



Pràctica 1: Cerca amb restriccions

Pol Pugibet Martinez - 1630568, Laia Rubio Castro - 1600830, Marc Serra Ortega - 1551550

Grup GM12:30_1

Coneixement Raonament i Incertesa

Grau en Enginyeria Informàtica

Escola d'Enginyeria

Índex

INTRODUCCIÓ	3
SOLUCIÓ PROPOSADA	3
IMPLEMENTACIÓ INICIAL	3
NIVELL C - CROSSWORD PUZZLE (BACKTRACKING)	4
NIVELL B - CROSSWORD PUZZLE (FORWARD CHECKING)	5
PROBLEMES SORGITS	6
CONCLUSIONS I MILLORES	6

Introducció

En aquesta pràctica es presenta el problema de la cerca amb restriccions. Se'ns proporciona un fitxer que conté 100 paraules diferents i un que conté un esquema d'un tauler de mots encreuats amb la finalitat d'emplenar-lo amb les paraules donades.

Els objectius assolits han estat:

- Assimilar els conceptes de cerca amb satisfacció de restriccions.
- Aplicar el model de backtracking amb forward checking explicat a teoria.
- Ser capaços d'aplicar la teoria en problemes reals.
- Validar els resultats obtinguts.
- Fomentar la capacitat per presentar resultats de forma adequada davant d'altres persones.

Els quals argumentarem a continuació a mesura que expliquem els diferents apartats, juntament amb els inconvenients sorgits i la metodologia que hem seguit per a solucionar-los.

Finalment exposarem la conclusió final i algunes possibles millores a implementar en el futur.

Solució proposada

El problema està organitzat en tres nivells de dificultat, per això explicarem els passos seguits a cadascun dels nivells per separat, simplificant així també el raonament de les decisions preses. Per a totes les implementacions hem reduït el bottleneck de cada funció per tal d'obtenir els resultats més òptims.

```
Backtracking(crossword_CB_v3.txt, diccionari_CB_v3.txt):  
1590 function calls (1579 primitive calls) in 0.004 seconds
```

```
ForwardChecking(crossword_CB_v3.txt, diccionari_CB_v3.txt):  
1475 function calls (1464 primitive calls) in 0.003 seconds
```

Tot i que la diferència amb aquest tamany de diccionari no és gaire notable, quan utilitzem un diccionari generat per nosaltres mateixos amb més de 1100 paraules ja es comença a veure la millora del ForwardChecking.

```
Backtracking(crossword_CB_v3.txt, diccionari_AA.txt):  
111762 function calls (110870 primitive calls) in 0.118 seconds
```

```
ForwardChecking(crossword_CB_v3.txt, diccionari_AA.txt):  
14266 function calls (14264 primitive calls) in 0.009 seconds
```

Implementació inicial

Un cop ja enteníem el problema així com el que es demanava i les limitacions de les possibles solucions vam començar a treballar en la implementació.

Per a carregar les dades vam crear les funcions següents:

```
def loadCrossword(file):
```

Aquesta funció rep un fitxer i llegeix cada línia d'aquest i agafa cada paraula per a agregar-les a una llista, la qual retorna com a numpy array.

```
def loadDictionary(file):
```

Aquesta funció rep un fitxer i llegeix cada paraula de cada línia per a agregar-la a un diccionari. El qual està definit per a que la 'key' sigui la longitud i el 'value' sigui la llista de paraules amb aquella longitud. Al acabar, retorna aquest diccionari.

Per a fer la cerca de variables vam crear les funcions següents:

```
def cercaVariablesHoritzontal(taulell, variables):
```

Aquesta funció rep el taulell i una llista de variables, les quals corresponen amb les posicions on es poden posicionar paraules. La funció va recorrent les files del taulell per a veure quines seqüències corresponen a caselles vàlides, es a dir, no son '#'. Si la longitud de la seqüència és major a 1 l'afegeix a la llista.

```
def cercaVariablesVertical(taulell, variables):
```

Aquesta funció segueix la mateixa dinàmica que la funció anterior, però en aquest cas recorre el taulell per columnes.

```
def cercaVariables(taulell, variables):
```

Aquesta funció combina les dos funcions anteriors per a trobar les posicions exactes (x,y) on s'ubiquen les caselles vàlides.

```
def calculaPosicionsVariable(variable):
```

Aquesta funció rep una variable que segueix l'estructura següent:

variable = [[pos_inicial], len, v/h]; vertical = 1, horitzontal = 0. Recorre un rang igual a la seva longitud per saber les possibles posicions d'una paraula dintre del taulell.

```
def intersectLetter(variable, word, assigned_variable, pos):
```

Aquesta funció rep la variable, la word (paraula que es vol introduir al taulell), la assigned_variable (paraula introduïda al taulell) i la pos (posició que s'està comprovant). I, verifica si la lletra de la paraula introduïda coincideix amb la lletra de la que es vol introduir en una posició concreta

```
def isValid(word, variable, assigned_variables):
```

Aquesta funció comprova que la paraula que es vol introduir no hagi sigut utilitzada prèviament i que interseccioni correctament amb la resta de paraules. Si això es compleix retorna True.

```
def trobaInterseccions(variables):
```

Calcula les posicions de les variables per així trobar les interseccions entre elles.

```
def printSolution(assigned_variables, taulell):
```

Rep les variables assignades, es a dir, les paraules introduïdes al taulell i el taulell. Mostra per pantalla el resultat final.

Nivell C - Crossword puzzle (backtracking)

En aquest apartat el que havíem de fer era aplicar l'algorisme de backtracking per a trobar quina era la paraula del vocabulari donat que anava a cada lloc.

Abans de començar amb la implementació de la solució vam contestar les següents preguntes:

1. Quines són les variables?
Caselles consecutives amb direcció vertical o horitzontal de longitud > 1
2. Quin és el domini de les variables?
Paraules del diccionari
3. Quines són les restriccions del problema?
 - Files i columnes han d'estar dins del taulell
 - Files i columnes > 1 casella
 - Interseccions de files i columnes tenen la mateixa lletra
 - Si es troba una # acaba la fila o la columna
 - Les paraules han d'estar escrites de dalt a baix i d'esquerra a dreta
 - No es pot repetir una paraula
 - Les paraules han de tenir una llargada menor o igual al màxim de m o n del taulell
4. Quin és el tamany de l'espai de solucions inicial?
 D^v on D es el domini de paraules del diccionari i v les variables
5. Has utilitzat alguna estratègia per reduir la profunditat mitjana de les branques?
Hem ordenat les variables de manera que es comenci primer per les variables amb més interseccions.

Un cop contestades les preguntes, i per tant, tenint encara més context del problema vam utilitzar les funcions implementades prèviament per a implementar finalment el Backtracking.

```
def backtracking(assigned_variables, variables, dictionary):
```

Nivell B - Crossword puzzle (forward checking)

En aquest apartat es demanava implementar el backtracking amb el forward checking.

Com al nivell anterior, abans de començar amb la implementació de la solució vam contestar les següents preguntes:

1. Quina és la diferència entre els dos mètodes?
La diferència és la forma en la que abordan el problema. Backtracking va avançant en la solució fins que troba una solució invàlida i aleshores torna enrere mentre que forward checking elimina directament les solucions invalides abans d'explorar-les.
2. Com millora aquest respecte l'anterior?
Aquest algorisme redueix l'espai de busqueda per no haver de fer tantes iteracions.
3. Heu canviat les variables, el domini o les restriccions del problema?
Tot i que hem utilitzat les mateixes variables i estructura de domini original, hem creat a mes a mes un nou diccionari de dominis que permet reduir la exploració i accedir millor a les variables.
4. Heu canviat l'estratègia utilitzada per reduir la profunditat mitjana de les branques utilitzada en l'apartat anterior?
Com expliquem més endavant, hem intentat implementar MRV però la nostra implementació no ha estat eficient.

En aquest nivell per a la implementació de la solució vam crear les funcions següents:

```
def initDomains(variables, dictionary):
```

Rep les variables (posicions on es poden col·locar les paraules) i el diccionari. Recorre cada element de variables i busca les paraules del diccionari amb la mateixa longitud i les afegeix al diccionari (fcDomains), retornant-lo al final.

```
def actualitzarDominis(variables, var, word, fcDomains, index, assigned_variables):
```

Comprova que la paraula que es vol introduir no hagi estat utilitzada prèviament, si la seva posició està en la llista d'interseccions per anteriors paraules, comprova que la intersecció concordi per a les possibles paraules solució. Si tot això es compleix, retorna True.

```
def calculateMRV(variables, assigned_variables, fcDomains):
```

Calcula per a cada variable no assignada el seu MRV (Menor Valor Restringit) com la longitud de domini del diccionari (fcDomains). Endreça la llista en ordre ascendent i la retorna.

Finalment, hem vist que la nostra implementació del MRV no era òptima, sinó que perjudicava al temps d'execució del nostre programa. Degut a això hem decidit deixar d'utilitzar-lo.

```
def forwardChecking(assigned_variables, variables, dictionary,  
fcDomains):
```

Implementa el forwardChecking.

Problemes sorgits

El major problema ha estat entendre com adaptar els models presentats al nostre cas. Després d'analitzar totes les parts i entendre realment el funcionament del problema, les seves restriccions i el resultat que volíem obtenir hem sigut capaços d'implementar una solució.

Conclusions i millores

Aquesta pràctica ens ha permet familiaritzar-nos més amb el funcionament dels diferents algorismes de cerca mitjançant la seva aplicació en un cas real. La cerca d'informació addicional ens ha permet tenir una millor perspectiva de la metodologia que podíem seguir per a resoldre els problemes ja que ens ha ajudat a entendre millor totes les parts. A més a més, al tractar-se d'un taulell de mots encreuat la complexitat del problema era adequada pel nostre nivell.

De cara a futur, podríem centrar-nos en millorar la qualitat del codi, ja que la nostra prioritat ha estat trobar la solució amb una bona complexitat algorítmica, però no necessàriament la millor.